NEXT ENGINEER · _SHEERAT_

Computer Organization and Architecture

BCS302

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 3 | **Credits:** 4

L-T-P: 3-1-0 | **Scheme:** AKTU B.Tech 2nd year, 2023-24 onwards

UNIT 1 Introduction, Buses and Addressing Modes

UNIT 2 Arithmetic and Logic Unit

UNIT 3 Control Unit

UNIT 4 Memory

UNIT 5 Input / Output

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: what you need to understand, with formulas and comparison tables.
3. **Solved examples**: the numericals that keep appearing (Booth, IEEE 754, cache mapping, pipelining, memory chips).
4. **Code to remember**: here, the register-transfer steps and algorithms you must reproduce.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 to 4 hours):** Units 1 and 2. Practise Booth's algorithm and IEEE 754 conversion until you can do them without looking. They are almost always asked.
- **Afternoon (3 hours):** Units 3 and 4. Cache mapping numericals and hardwired vs microprogrammed control are the big-mark topics.
- **Evening (2 hours):** Unit 5 (DMA, interrupts, I/O modes), then the "Last-hour checklist".
- **In the exam:** draw block diagrams for everything (bus, CPU, cache, DMA). Show every step of a numerical in a table. Write formulas before substituting values.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction, Buses and Addressing Modes

✍️ 2-mark definitions

- **Computer organization vs architecture:** architecture is what the programmer sees (instruction set, addressing modes, data types); organization is how the hardware implements it (control signals, memory technology, interfaces).
- **Functional units:** input, output, memory, arithmetic and logic unit (ALU) and control unit. ALU + control unit + registers form the CPU.
- **Bus:** a shared group of wires that carries information between units. A system bus has three parts: **address bus** (one-way, CPU to memory/I/O), **data bus** (two-way) and **control bus** (read, write, interrupt, clock signals).
- **Bus arbitration:** deciding which of several devices (bus masters) gets control of the bus when more than one requests it.
- **Register transfer language (RTL):** notation for micro-operations, e.g. `R2 ← R1` copies R1 into R2; `P: R2 ← R1` does it only when control signal P = 1.
- **Addressing mode:** the rule that tells the CPU how to find the operand (effective address) from the address field of an instruction.
- **Effective address (EA):** the actual memory address of the operand after the addressing mode is applied.
- **Stack pointer (SP):** a register that holds the address of the top of the stack.

💡 Core concepts

Bus types

Type	Meaning
Single bus	All units share one bus. Cheap but only one transfer at a time.
Multiple bus	Separate buses (e.g. memory bus, I/O bus). Faster, costs more.
Synchronous bus	Transfers are timed by a common clock.
Asynchronous bus	Transfers use handshaking signals (ready/acknowledge), no common clock.

Bus arbitration methods

Method	How it works	Pro / Con
Daisy chaining (serial)	Bus-grant signal passes from device to device; the first requesting device keeps it	Simple, few wires; fixed priority, slow, one failure breaks the chain
Polling	Controller checks devices one by one using a count on poll lines	Priority can be changed; slower
Independent requests (parallel)	Each device has its own request and grant lines to the arbiter	Fastest; many wires

Arbitration can be **centralized** (one arbiter) or **distributed** (devices decide among themselves).

Bus transfer using multiplexers: for k registers of n bits each, you need **n multiplexers of k×1** each; select lines choose the register that drives the bus. Using three-state buffers is the alternative.

Memory transfer: read `DR ← M[AR]`; write `M[AR] ← R1`.

General register organization: registers R1...R7 connected to the ALU through two multiplexers (A-bus and B-bus selectors) and a decoder that selects the destination. A **control word** has fields SELA, SELB, SELD and OPR. Example: `R1 ← R2 + R3` sets SELA = R2, SELB = R3, SELD = R1, OPR = ADD.

Stack organization

- Register stack (e.g. 64 words): push `SP ← SP + 1, M[SP] ← DR`; pop `DR ← M[SP], SP ← SP - 1`. Flags FULL and EMPTY.
- Memory stack grows toward lower addresses in many machines: push `SP ← SP - 1, M[SP] ← DR`; pop `DR ← M[SP], SP ← SP + 1`.
- Stack machines use **zero-address instructions** and evaluate expressions in **reverse Polish (postfix)** form.

Addressing modes (learn this table)

Mode	Effective address / operand	Use
Implied	Operand is implicit (e.g. `CMA`, stack ops)	Short instructions
Immediate	Operand is in the instruction itself	Constants
Register	Operand is in a register	Fast
Register indirect	EA = contents of register	Pointers
Autoincrement / autodecrement	Like register indirect, then register is incremented (or decremented first)	Arrays, stacks
Direct	EA = address field	Simple variables
Indirect	EA = M[address field]	Pointers in memory
Relative	EA = PC + address field	Branches; position-independent code
Indexed	EA = index register + address field	Arrays
Base register	EA = base register + address field	Relocation of programs

🧮 Solved examples

Example 1 (addressing modes). A two-word instruction is stored at addresses 200 and 201, with address field = 500 in word 201. After fetching it, PC = 202. R1 = 400, index register XR = 100, and memory location 500 contains 800. Find the effective address for each mode.

Mode	Effective address
Immediate	201 (the operand 500 is in the instruction itself)
Direct	500

Mode	Effective address
Indirect	M[500] = 800
Relative	PC + 500 = 202 + 500 = 702
Indexed	XR + 500 = 100 + 500 = 600
Register	none: the operand is R1 itself (400)
Register indirect	400
Autoincrement	400 (then R1 becomes 401)
Autodecrement	399 (R1 is decremented first)

Example 2 (instruction formats). Evaluate X = (A + B) × (C + D).

- **Three-address:** `ADD R1, A, B` · `ADD R2, C, D` · `MUL X, R1, R2`
- **Two-address:** `MOV R1, A` · `ADD R1, B` · `MOV R2, C` · `ADD R2, D` · `MUL R1, R2` · `MOV X, R1`
- **One-address (accumulator):** `LOAD A` · `ADD B` · `STORE T` · `LOAD C` · `ADD D` · `MUL T` · `STORE X`
- **Zero-address (stack):** `PUSH A` · `PUSH B` · `ADD` · `PUSH C` · `PUSH D` · `ADD` · `MUL` · `POP X`

Fewer addresses → shorter instructions but longer programs.

Example 3 (bus with multiplexers). A bus system for 8 registers of 16 bits each needs 16 multiplexers, each 8×1, with 3 select lines.

Register transfers to remember

```
Fetch:        T0: AR ← PC
              T1: IR ← M[AR], PC ← PC + 1
Decode:       T2: decode IR, AR ← address field of IR
Push (reg.):  SP ← SP + 1;  M[SP] ← DR;  if SP = 0 then FULL ← 1;  EMPTY ← 0
Pop  (reg.):  DR ← M[SP];  SP ← SP - 1;  if SP = 0 then EMPTY ← 1;  FULL ← 0
```

⭐ Important questions

1. Draw and explain the functional units of a digital computer and how they are connected by the system bus. *Block diagram + role of address, data and control buses.*
2. What is bus arbitration? Explain daisy chaining, polling and independent request methods. *Diagram for each, with a pro/con line.*
3. Explain all addressing modes with examples, and solve a numerical like Example 1. *Table plus calculation.*
4. Explain general register organization with a block diagram and control word format. *Show one micro-operation encoded as a control word.*
5. Explain register stack and memory stack organization with push/pop micro-operations.
6. Write a program for an expression using three-, two-, one- and zero-address instructions.

UNIT 2 Arithmetic and Logic Unit

✍️ 2-mark definitions

- **Ripple-carry adder:** n full adders chained so each carry waits for the previous one; delay grows with n.
- **Carry look-ahead adder (CLA):** computes all carries in parallel from generate and propagate signals, removing the ripple delay.
- **Generate (G_i) / propagate (P_i):** $G_i = A_iB_i$ (this stage creates a carry); $P_i = A_i \oplus B_i$ (this stage passes an incoming carry on).
- **Booth's algorithm:** a method for multiplying signed (2's complement) numbers that skips over runs of 1s, reducing additions.
- **Array multiplier:** a combinational multiplier that forms all partial products with AND gates and adds them with an array of adders.
- **Overflow:** when the result of signed arithmetic does not fit in the given bits; detected when the carry into the sign bit differs from the carry out of it.
- **Floating-point number:** a number stored as sign, exponent and mantissa (fraction): value = $(-1)^s$ × 1.M × 2^(E − bias).
- **Normalization:** shifting the mantissa so its leading bit is 1 (1.xxx form), adjusting the exponent to match.
- **Bias:** a constant added to the true exponent so it can be stored as an unsigned number (127 for single, 1023 for double precision).

💡 Core concepts

Carry look-ahead equations (4-bit)

- $C_1 = G_0 + P_0C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0C_0$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$
- $C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$
- Sum $S_i = P_i \oplus C_i$. All carries come out after about 3 gate delays, whatever the position.

Booth's algorithm rules (look at Q_0 and Q_{-1}, the bit to its right)

$Q_0\ Q_{-1}$	Action
0 0	Only arithmetic shift right
1 1	Only arithmetic shift right
1 0	A ← A − M, then shift right
0 1	A ← A + M, then shift right

Repeat n times (n = number of bits). The answer is in A and Q together. Arithmetic shift right keeps the sign bit of A.

Array multiplier (n × n bits): needs n^2 AND gates for the partial products and about $n(n-1)$ full/half adders to add them. A 4 × 4 multiplier uses 16 AND gates and 12 adders (three rows of 4).

Division

- **Restoring:** shift A,Q left; A ← A − M; if A < 0 then $Q_0 \leftarrow 0$ and restore A ← A + M, else $Q_0 \leftarrow 1$. Repeat n times. Quotient in Q, remainder in A.
- **Non-restoring:** avoids the restore step. If A ≥ 0, shift and subtract M; if A < 0, shift and add M. Then set Q_0 from the sign of A. A final correction (add M) is needed if the remainder ends negative.

IEEE 754 formats

Format	Sign	Exponent	Mantissa	Bias
Single (32-bit)	1	8	23	127
Double (64-bit)	1	11	52	1023

Special values: exponent all 0s and mantissa 0 → **zero**; exponent all 1s and mantissa 0 → **infinity**; exponent all 1s and mantissa ≠ 0 → **NaN**; exponent all 0s and mantissa ≠ 0 → **denormal** numbers.

Floating-point addition/subtraction steps: (1) check for zeros, (2) align mantissas by shifting the smaller number right until exponents match, (3) add or subtract mantissas, (4) normalize the result and round. **Multiplication:** add exponents (subtract one bias), multiply mantissas, normalize. **Division:** subtract exponents (add bias back), divide mantissas, normalize.

ALU design: an ALU stage combines an arithmetic circuit (adder with input selection: add, subtract, increment, decrement, transfer) and a logic circuit (AND, OR, XOR, NOT), with a multiplexer choosing the output according to select lines.

🧮 Solved examples

Example 1 (Booth's algorithm): 7 × (−3) using 4 bits. M = 0111, −M = 1001, Q = 1101 (−3), A = 0000, Q_{-1} = 0.

Step	Q_0Q_{-1}	Operation	A	Q	Q_{-1}
Start			0000	1101	0
1	10	A ← A − M	1001	1101	0
		Shift right	1100	1110	1
2	01	A ← A + M	0011	1110	1
		Shift right	0001	1111	0
3	10	A ← A − M	1010	1111	0
		Shift right	1101	0111	1
4	11	Shift right only	1110	1011	1

Result A Q = **1110 1011** = −(0001 0101) = **−21**. ✔

Example 2 (IEEE 754 single precision): represent −12.375.

- 12 = 1100, 0.375 = 0.011 → 12.375 = 1100.011_2 = 1.100011×2^3.
- Sign = 1. Exponent = 3 + 127 = 130 = 10000010. Mantissa = 100011 followed by zeros (23 bits).
- Result: **1 10000010 10001100000000000000000** = **C1460000** (hex).

Example 3 (restoring division): 7 ÷ 3 with 4 bits. M = 0011, Q = 0111, A = 0000.

Cycle	After shift (A, Q)	A − M	Result
1	0000, 111_	1101 (negative)	Q_0 = 0, restore → A = 0000, Q = 1110
2	0001, 110_	1110 (negative)	Q_0 = 0, restore → A = 0001, Q = 1100
3	0011, 100_	0000 (≥ 0)	Q_0 = 1 → A = 0000, Q = 1001
4	0001, 001_	1110 (negative)	Q_0 = 0, restore → A = 0001, Q = 0010

Quotient Q = **0010 (2)**, remainder A = **0001 (1)**. ✔

Example 4 (overflow). 4-bit signed: 0110 (+6) + 0101 (+5) = 1011, which reads as −5. Carry into the sign bit = 1, carry out = 0. They differ → **overflow**.

Algorithm steps to remember

```
Booth (n bits):   A ← 0, Q-1 ← 0, count ← n
                  repeat: if Q0Q-1 = 10 then A ← A - M
                          if Q0Q-1 = 01 then A ← A + M
                          arithmetic shift right A, Q, Q-1;  count ← count - 1
                  until count = 0;  product = A:Q
Restoring div.:   A ← 0;  repeat n times:
                          shift A:Q left;  A ← A - M
                          if A < 0 then Q0 ← 0, A ← A + M  else Q0 ← 1
```

⭐ Important questions

1. Explain Booth's multiplication algorithm with a flowchart and multiply two signed numbers. *Flowchart + full step table like Example 1.*
2. Design a 4-bit carry look-ahead adder. Derive C_1 to C_4 and compare with a ripple-carry adder. *Equations + block diagram + delay comparison.*
3. Represent a number in IEEE 754 single and double precision. *Show binary conversion, normalization, biased exponent, final bits and hex.*
4. Explain restoring (and non-restoring) division with a flowchart and an example.
5. Explain the array multiplier for 4 × 4 bits with a diagram.
6. Explain floating-point addition/subtraction with a flowchart.

UNIT 3 Control Unit

✍️ 2-mark definitions

- **Instruction cycle:** the steps the CPU repeats for each instruction: fetch, decode, (fetch operand / indirect), execute, and check for interrupts.
- **Micro-operation:** an elementary operation done on data in registers in one clock pulse, e.g. shift, count, clear, load.
- **Control unit:** the part of the CPU that generates timing and control signals to carry out instructions.
- **Hardwired control:** control signals are produced by fixed logic circuits (gates, flip-flops, decoders).
- **Microprogrammed control:** control signals are stored as control words (microinstructions) in a control memory and read out in sequence.
- **Control memory:** a fast memory (usually ROM) holding the microprogram.
- **Control address register (CAR):** holds the address of the next microinstruction. **Control data register (CDR):** holds the microinstruction just read.
- **Pipelining:** overlapping the execution of several instructions by splitting work into stages, like an assembly line.
- **Hazard:** a situation that stops the next instruction in a pipeline from executing in its clock cycle.
- **RISC:** reduced instruction set computer: few, simple, fixed-length instructions, mostly one per clock cycle.

💡 Core concepts

Instruction types: data transfer (MOV, LOAD, STORE, PUSH, POP), data manipulation (arithmetic, logical, shift), and program control (branch, jump, call, return, compare, test).

Program control and status flags: C (carry), S (sign), Z (zero), V (overflow). Conditional branches test these. **Subroutine call:** save the return address (usually on the stack), then jump; **return:** pop it back into PC.

Hardwired vs microprogrammed control (very frequent question)

Hardwired	Microprogrammed
Signals from logic circuits	Signals from microinstructions in control memory
Fast	Slower (memory access per microinstruction)
Hard to modify or extend	Easy to change: rewrite the microprogram
Costly for complex instruction sets	Cheaper for large instruction sets
Used in RISC processors	Used in CISC processors

Horizontal vs vertical microprogramming

Horizontal	Vertical
One bit per control signal; long microinstructions	Signals encoded in fields; short microinstructions
No decoding needed, very fast	Needs decoders, slower
High parallelism	Less parallelism
Bigger control memory	Smaller control memory

Microprogram sequencing: the next microinstruction address comes from (1) incrementing CAR, (2) a branch address in the microinstruction (conditional or unconditional), (3) mapping from the instruction's opcode, or (4) a subroutine return register. A microinstruction format often has fields: micro-operations F1 F2 F3, condition CD, branch BR, address AD.

Pipelining

- Speed-up for n tasks on a k-stage pipeline: **S = n·k / (k + n − 1)**. As n → ∞, S → k.
- Time for n tasks: (k + n − 1) × clock period.
- **Hazards:**
- **Structural:** two instructions need the same hardware (e.g. one memory port). Fix: duplicate resources.
- **Data:** an instruction needs a result not yet produced. Fix: forwarding (bypassing), stalls, compiler reordering.
- **Control (branch):** the next instruction is unknown until a branch resolves. Fix: branch prediction, delayed branch, prefetching both paths.
- Typical 4-stage instruction pipeline: Fetch (FI), Decode (DA), Fetch operand (FO), Execute (EX).

RISC vs CISC

RISC	CISC
Few, simple instructions	Many complex instructions
Fixed-length instructions	Variable-length instructions
Few addressing modes	Many addressing modes
Load/store architecture: only LOAD/STORE touch memory	Many instructions access memory
Many registers	Fewer registers
Hardwired control, heavy pipelining	Usually microprogrammed
Example: ARM, MIPS	Example: Intel x86

🧮 Solved examples

Example 1 (pipeline speed-up). A 4-stage pipeline with a 20 ns clock runs 100 instructions.

- Pipelined time = (k + n − 1) × t = (4 + 100 − 1) × 20 = 103 × 20 = **2060 ns**.
- Non-pipelined time = n × k × t = 100 × 4 × 20 = **8000 ns**.
- Speed-up = 8000 / 2060 ≈ **3.88** (close to the ideal of 4).

Example 2 (pipeline space-time diagram, 4 stages, 5 instructions).

```
Clock:   1    2    3    4    5    6    7    8
I1:     FI   DA   FO   EX
I2:          FI   DA   FO   EX
I3:               FI   DA   FO   EX
I4:                    FI   DA   FO   EX
I5:                         FI   DA   FO   EX
```

All 5 instructions finish in k + n − 1 = 4 + 5 − 1 = **8 cycles** instead of 20.

Example 3 (control memory size). A microprogrammed control unit has 128 microinstructions, each 24 bits wide. Control memory = 128 × 24 bits; CAR needs $\log_2 128$ = **7 bits**.

Instruction cycle to remember

```
Fetch:     T0: AR ← PC
           T1: IR ← M[AR], PC ← PC + 1
Decode:    T2: D0…D7 ← decode IR(12-14), AR ← IR(0-11), I ← IR(15)
Indirect:  T3: if I = 1 then AR ← M[AR]
Execute:   T4 onward, depends on opcode, e.g. ADD:  DR ← M[AR];  AC ← AC + DR, SC ← 0
Interrupt: if IEN = 1 and an interrupt is pending: save PC, PC ← interrupt vector
```

⭐ Important questions

1. Explain the instruction cycle with a flowchart, including the interrupt cycle. *Fetch, decode, indirect, execute, interrupt with RTL for each.*
2. Compare hardwired and microprogrammed control units with block diagrams. *Table + both diagrams.*
3. Explain horizontal and vertical microprogramming and microprogram sequencing. *Microinstruction format and next-address logic.*
4. What is pipelining? Explain pipeline hazards and how to resolve them, and calculate speed-up. *Space-time diagram + formula + numerical.*
5. Compare RISC and CISC architectures.
6. Explain instruction formats and types of instructions with examples.

UNIT 4 Memory

✍️ 2-mark definitions

- **Memory hierarchy:** arranging memories from small, fast, costly (registers, cache) to large, slow, cheap (main memory, disk, tape) so the system is both fast and affordable.
- **Locality of reference:** programs tend to reuse recently used data (**temporal locality**) and data near it (**spatial locality**). This is why caches work.
- **SRAM vs DRAM:** SRAM stores bits in flip-flops (fast, no refresh, costly, used for cache); DRAM stores bits as charge on capacitors (slower, needs refresh, dense and cheap, used for main memory).
- **ROM types:** ROM (fixed at manufacture), PROM (programmed once), EPROM (erased with UV light), EEPROM (erased electrically), flash (EEPROM erased in blocks).
- **Cache memory:** a small, fast memory between CPU and main memory that holds copies of recently used blocks.
- **Hit ratio (h):** fraction of memory accesses found in the cache.
- **Cache mapping:** the rule deciding where a main memory block can be placed in the cache: direct, associative or set-associative.
- **Write-through vs write-back:** write-through updates main memory on every write; write-back updates only the cache and writes the block back when it is replaced (uses a dirty bit).
- **Virtual memory:** a technique that lets programs use an address space larger than physical memory, by keeping parts on disk and loading pages on demand.
- **Page fault:** when the referenced page is not in main memory and must be brought from disk.
- **TLB:** translation look-aside buffer, a small fast cache of recent page-table entries.

💡 Core concepts

2D vs 2½D memory organization

2D	2½D
One decoder selects a whole word (row); words arranged as rows	Address split into row and column parts with two smaller decoders
Simple but the decoder gets huge for large memories	Far fewer decoder outputs: e.g. 1024 words need 32 + 32 lines instead of 1024
Each word line drives a full word	Each bit plane selected by row and column (coincident selection)

Cache mapping

Technique	Where a block can go	Address fields	Pros / cons
Direct	Exactly one line: line = block mod (number of lines)	Tag, Line (index), Word	Simple, cheap; conflict misses

Technique	Where a block can go	Address fields	Pros / cons
Fully associative	Any line	Tag, Word	Flexible, fewest misses; costly comparators
k-way set-associative	Any line within one set: set = block mod (number of sets)	Tag, Set, Word	Balance of both; most common

Replacement policies: FIFO, LRU (least recently used), LFU (least frequently used), random. Direct mapping needs no policy.

Average memory access time

- Simultaneous (look-aside) access: $T = h \cdot Tc + (1 - h) \cdot Tm$
- Hierarchical (look-through) access: $T = h \cdot Tc + (1 - h) \cdot (Tc + Tm)$

Auxiliary memory

- **Magnetic disk:** platters → tracks → sectors; cylinders = same track on all surfaces. **Access time = seek time + rotational latency + transfer time.** Average rotational latency = half a rotation.
- **Magnetic tape:** sequential access, cheap, used for backup.
- **Optical disks:** CD, DVD, Blu-ray; data stored as pits and lands read by a laser; single spiral track.

Virtual memory: a virtual address = page number + offset. The **page table** maps page number to frame number; the **TLB** caches recent mappings. On a page fault the OS loads the page from disk, possibly replacing another (FIFO, LRU, optimal). **Segmentation** divides the program into variable-size logical segments instead of fixed pages.

🧮 Solved examples

Example 1 (cache mapping fields). Main memory = 64 KB, cache = 2 KB, block = 16 bytes, byte addressable.

- Address bits = $\log_2(64K)$ = **16**. Word (offset) = $\log_2 16$ = **4 bits**.
- Cache lines = 2 KB / 16 B = 128.
- **Direct:** line = $\log_2 128$ = 7 bits; tag = 16 − 7 − 4 = **5 bits** → Tag 5 | Line 7 | Word 4.
- **Fully associative:** tag = 16 − 4 = **12 bits** → Tag 12 | Word 4.
- **4-way set-associative:** sets = 128 / 4 = 32 → set = 5 bits; tag = 16 − 5 − 4 = **7 bits** → Tag 7 | Set 5 | Word 4.

Example 2 (average access time). Cache access = 10 ns, main memory access = 100 ns, hit ratio = 0.9.

- Simultaneous: 0.9 × 10 + 0.1 × 100 = 9 + 10 = **19 ns**.
- Hierarchical: 0.9 × 10 + 0.1 × (10 + 100) = 9 + 11 = **20 ns**.

Example 3 (building memory from chips). Make a 4K × 16 memory from 1K × 8 chips.

- Chips = (4K / 1K) × (16 / 8) = 4 × 2 = **8 chips**, arranged as 4 rows of 2.
- Address lines = $\log_2 4K$ = 12: the lower 10 go to every chip; the upper 2 drive a **2-to-4 decoder** that selects one row.

Example 4 (page replacement, 3 frames). Reference string: 7 0 1 2 0 3 0 4.

Ref	7	0	1	2	0	3	0	4
FIFO	F	F	F	F (out 7)	hit	F (out 0)	F (out 1)	F (out 2)
LRU	F	F	F	F (out 7)	hit	F (out 1)	hit	F (out 2)

FIFO: **7 faults**. LRU: **6 faults**.

Example 5 (disk). A disk spins at 7200 rpm with average seek time 8 ms. One rotation = 60/7200 s = 8.33 ms, so average rotational latency = **4.17 ms**. Average access time (ignoring transfer) ≈ 8 + 4.17 = **12.17 ms**.

Formulas to remember

```
Cache lines         = cache size / block size
Sets                = lines / k              (k-way)
Word bits           = log2(block size)
Direct:             tag = address bits - line bits - word bits
Set-associative:    tag = address bits - set bits - word bits
Fully associative:  tag = address bits - word bits
T(simultaneous)     = h·Tc + (1 - h)·Tm
T(hierarchical)     = h·Tc + (1 - h)·(Tc + Tm)
Chips needed        = (total words / chip words) × (word width / chip width)
Disk access         = seek + rotational latency + transfer
```

⭐ Important questions

1. Explain the memory hierarchy with a diagram, and why it works (locality of reference).
2. Explain direct, associative and set-associative cache mapping with diagrams, and solve an address-field numerical. *All three formats like Example 1.*
3. Explain 2D and 2½D memory organization with diagrams.
4. Explain cache write policies and replacement algorithms, and find the hit ratio / average access time.
5. What is virtual memory? Explain paging, page table, TLB and page replacement with an example.
6. Explain magnetic disk structure and access time, with a numerical.
7. Design a memory of given size using smaller RAM chips with a decoder.

UNIT 5 Input / Output

✍️ 2-mark definitions

- **Peripheral device:** an input, output or storage device attached to the computer (keyboard, monitor, printer, disk).
- **I/O interface:** circuitry between the system bus and a peripheral that handles differences in speed, data format and signals.
- **I/O port:** a register in the interface that the CPU reads or writes to talk to a device (data, status and control registers).
- **Isolated vs memory-mapped I/O:** isolated I/O uses separate address space and special IN/OUT instructions; memory-mapped I/O gives devices ordinary memory addresses, so normal load/store instructions work.
- **Interrupt:** a signal from a device or condition that makes the CPU pause the current program and run an interrupt service routine (ISR).
- **Exception:** an interrupt caused inside the CPU by the running program (divide by zero, invalid opcode, page fault).
- **Vectored interrupt:** the interrupting device supplies the address (vector) of its ISR.
- **DMA (direct memory access):** a DMA controller transfers a block of data directly between I/O and memory without the CPU handling each word.
- **I/O processor (IOP) / channel:** a special processor that executes I/O programs on its own, freeing the CPU.
- **Synchronous vs asynchronous serial transfer:** synchronous sends data with a shared clock in continuous blocks; asynchronous sends each character framed by start and stop bits.

💡 Core concepts

Modes of data transfer

Mode	How it works	CPU involvement
Programmed I/O	CPU keeps checking the device status flag (polling)	Very high: CPU waits
Interrupt-initiated I/O	Device interrupts the CPU when ready; CPU runs ISR	Medium: no waiting, but CPU moves each word
DMA	DMA controller moves a whole block directly to/from memory	Low: CPU only sets up and gets one interrupt at the end

DMA working: CPU loads the DMA controller with the memory start address, word count and direction. The controller sends **bus request (BR)**; the CPU finishes the current bus cycle and replies **bus grant (BG)**, floating its buses. The controller transfers data, then interrupts the CPU when the count reaches zero.

- **Burst mode:** the whole block in one go; CPU waits.

- **Cycle stealing:** one word per stolen bus cycle, interleaved with CPU work.
- **Transparent mode:** transfers only when the CPU is not using the bus.

Interrupts

- **Types:** external/hardware (I/O devices, timer, power failure), internal (exceptions/traps from the program), and software interrupts (instructions like `INT` /supervisor call).
- **Maskable vs non-maskable:** maskable can be disabled by the CPU; non-maskable (NMI) cannot, used for critical events.
- **Priority handling:** polling (software checks devices in priority order), daisy chain (hardware serial priority), parallel priority interrupt (priority encoder + mask register).
- **Interrupt cycle:** finish the current instruction, save PC (and status) on the stack, load PC with the ISR address, disable further interrupts; at the end the ISR returns and restores PC.

Asynchronous data transfer

- **Strobe control:** one control line signals when data is valid. Simple, but there is no confirmation the receiver got it.
- **Handshaking:** two lines (data valid / data accepted) give confirmation in both directions. Reliable.

Serial communication: asynchronous frame = 1 start bit (0) + 5 to 8 data bits + optional parity + 1 or 2 stop bits (1). A **UART** does asynchronous serial transfer.

Standard communication interfaces: RS-232 (serial), USB (universal serial bus, hot-pluggable, powers devices), PCI/PCIe (expansion bus), SATA (storage), parallel printer port, Ethernet.

🧮 Solved examples

Example 1 (asynchronous transfer rate). A line sends characters with 1 start bit, 8 data bits, 1 parity bit and 2 stop bits at 1200 bps (baud). Bits per character = 12, so the line carries 1200 / 12 = **100 characters per second**.

Example 2 (DMA vs interrupt, idea). Transferring 1000 words with interrupt-driven I/O means 1000 interrupts, each with save/restore overhead. With DMA the CPU sets up the transfer once and gets **one** interrupt at the end, which is why DMA is used for disks.

Sequences to remember

```
Programmed I/O:   loop: read status; if flag = 0 goto loop;  read data;  store;  repeat
Interrupt cycle:  SP ← SP - 1;  M[SP] ← PC;  PC ← ISR address;  IEN ← 0
DMA:              CPU → DMA: address, count, direction;  DMA → CPU: BR;  CPU → DMA: BG;
                  DMA transfers words;  count = 0 → interrupt CPU
Handshake (source-initiated):  source puts data, raises "data valid" → destination reads,
                  raises "data accepted" → source drops "data valid" → destination drops
"accepted"
```

⭐ Important questions

1. Explain the modes of data transfer: programmed I/O, interrupt-initiated I/O and DMA. Compare them. *Diagram for each + comparison table.*
2. What is DMA? Explain the DMA controller with a block diagram and the BR/BG sequence. *Include burst and cycle-stealing modes.*
3. What is an interrupt? Explain types of interrupts, interrupt hardware and priority (daisy chain / parallel priority).
4. Explain strobe and handshaking methods of asynchronous data transfer with timing diagrams.
5. Explain synchronous and asynchronous serial communication, and a standard interface (e.g. USB or RS-232).
6. Write short notes on I/O processors and I/O channels.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ Address, data and control buses; three bus arbitration methods.
- ☐ Addressing modes table, and the EA numerical.
- ☐ Three/two/one/zero-address code for an expression.
- ☐ CLA equations C_1 to C_4.
- ☐ Booth's rules (10 → subtract, 01 → add) and one full worked table.
- ☐ IEEE 754: bias 127/1023, 1-8-23 and 1-11-52 bit layouts.
- ☐ Restoring division steps.
- ☐ Hardwired vs microprogrammed; horizontal vs vertical.
- ☐ Pipeline speed-up S = nk / (k + n − 1); three hazards.
- ☐ RISC vs CISC table.
- ☐ Cache mapping field formulas; average access time formulas.
- ☐ SRAM vs DRAM; 2D vs 2½D.
- ☐ FIFO vs LRU page replacement.
- ☐ Programmed I/O vs interrupt vs DMA; BR/BG.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Computer System Architecture	M. Morris Mano	Pearson	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
2	Computer Organization (5th ed.)	Carl Hamacher, Zvonko Vranesic, Safwat Zaky	McGraw-Hill	WorldCat
3	Computer Architecture and Organization (3rd ed.)	John P. Hayes	Tata McGraw-Hill	WorldCat
4	Computer Organization and Architecture: Designing for Performance (7th ed.)	William Stallings	Pearson Education	WorldCat
5	Computer Architecture	Behrooz Parhami	Oxford University Press	WorldCat
6	Computer Architecture: A Quantitative Approach (5th ed.)	John L. Hennessy, David A. Patterson	Elsevier	WorldCat
7	Structured Computer Organization	Andrew S. Tanenbaum	PHI	WorldCat

Official syllabus: AKTU B.Tech 2nd year CSE syllabus, 2023-24 scheme (PDF)

NEXT ENGINEER _sheerat_